



I haven't worked on my project in a long time. I need a refresher on how my final program is supposed to work.

Here's the end-to-end flow of your final Finance RAG system, in plain language:

1. Ingestion (offline, run once or when data changes)

- You feed in raw 10-K and earnings-call HTML files.
- A parser extracts the main text and discards navigation, headers, footers, etc.
- The text is normalized (cleaned, whitespace fixed).
- The document is split into chunks (e.g., ~500 tokens each, with overlap).
- Each chunk gets metadata: `document_id`, `chunk_id`, `company`, `ticker`, `document_type`, `fiscal_period`, `source_uri`, `section`, etc.
- Each chunk is embedded using your chosen embedding model.
- The embedding + metadata is stored as a point in Qdrant.

Result: a searchable vector index of finance document chunks.

2. Query time (when a user asks a question)

1. User sends a question to your FastAPI /ask endpoint.
2. Question validation: check length, format, and basic sanity.
3. Embed the question using the same embedding model as your chunks.
4. Retrieve top-k chunks from Qdrant using vector similarity (and optional filters like ticker or document type).
5. Check evidence quality:
 - If no results or very low similarity → abstain ("I don't have enough evidence...").
6. Generate an answer:
 - Send the retrieved chunks + question to the LLM.
 - Instruct the LLM to answer only from the provided context and cite each claim with chunk/document info.
7. Validate citations:
 - Ensure every citation in the answer refers to a retrieved chunk.
 - If not, treat as invalid → abstain or error.

8. Return structured response:

```
{
  "answer": "...",
  "citations": [...],
  "abstained": false,
  "retrieved_chunks": 5,
  "latency_ms": 342
}
```

9. Log everything:

- Request ID, latencies, result count, top score, abstention status, errors.

3. Evaluation (offline, run after changes)

- You have a fixed set of 20–30 questions with expected answers and relevant chunk IDs.
- For each question:
 - Run retrieval → measure Recall@k, Precision@k, MRR.
 - Run generation → measure faithfulness, citation accuracy, answer correctness.
- Aggregate metrics and compare across configurations (chunk size, k, thresholds).

4. Deployment

- Everything runs in Docker:
 - One container for your FastAPI app.
 - One container for Qdrant with persistent storage.
- You can start the system with `docker-compose up`.
- A README explains how to:
 - Ingest data.
 - Start the API.
 - Run evaluation.
 - Send example queries.

If you want, I can walk you through re-implementing this step-by-step from scratch.